

Universidad Técnica Federico Santa María

Departamento de Ingeniería Electrónica

Diseño FPGA con MATLAB

Informe de ejemplo



Autor Ejemplar

Profesor Guía

Valparaíso, 16 de enero de 2026

Resumen

La utilización de descripciones de alto nivel para el prototipado rápido de algoritmos es una práctica recurrente tanto en la industria como academia, debido a la abstracción y herramientas proporcionadas que facilitan el desarrollo, depuración y validación de algoritmos. Sin embargo, la gran mayoría de prototipos precisan ser implementados en sistemas físicos que operan con plataformas embebidas que no cuentan con los recursos para ejecutar descripciones de alto nivel. En consecuencia, una herramienta que facilite y reduzca la brecha que existe entre el traspaso de descripciones de alto nivel a sistemas embebidos, trae mejoras en los tiempos de desarrollo y productividad de quien las utiliza, permitiendo además reducir al error humano al sistematizar este proceso.

En particular, dado que el enfoque de esta memoria está en ser un aporte en proyectos del ámbito académico, es que se centra en explorar la utilidad efectiva de HDL Coder y HDL Verifier, herramientas que entrega Matlab para la generación automática de HDL a partir de descripciones realizadas en scripts de código o diagramas de bloques y dentro de un flujo de High-Level Synthesis (HLS). En específico, en esta memoria se propone y valida una metodología sistemática para la utilización e implementación de los códigos generados por HDL Coder, utilizando como ejemplos distintos casos de estudio que son implementados y ejecutados en FPGAs con diferentes sistemas de desarrollo y entornos de verificación. Para uno de los casos de estudio, particularmente relevante para proyectos en curso en el Departamento de Electrónica, se evalúan los tiempos de ejecución y la verificación obtenidos para la ejecución del código generado en cada plataforma. Los códigos fuente e información necesaria para reproducir y extender los experimentos reportados en este informe se encuentran disponibles en un repositorio de código.

Dado los resultados experimentales obtenidos a lo largo del desarrollo de esta memoria, es posible concluir que HDL Coder, apoyado por HDL Verifier, es una herramienta útil para la generación de código HDL optimizado para FPGAs con una equivalencia funcional a descripciones descritas en Matlab.

Índice general

Resumen	1
1. Introducción	4
1.1. Motivación y contexto.....	4
1.2. Planteamiento del problema	5
1.3. Metodología general.....	5
1.4. Alcances y contribuciones	6
1.5. Organización del informe.....	7
2. Antecedentes	8
2.1. Síntesis de alto nivel y herramientas de generación de código en la industria ..	8
2.2. Herramientas de MATLAB para generación y verificación de código	9
2.2.1. MATLAB Coder	9
2.2.2. Simulink Coder.....	9
2.2.3. Embedded Coder	10
2.2.4. HDL Coder	10
2.2.5. HDL Verifier.....	10
2.2.6. Discusión	11
3. Metodología y desarrollo	12
3.1. Aplicaciones de ejemplo	12
3.2. Flujo de trabajo	12
3.3. Plataformas objetivo y configuraciones	12
3.4. Implementaciones	13
4. Resultados experimentales	14
4.1. Validación funcional de códigos generados	14
4.1.1. Producto punto (flujo base)	14
4.1.2. PID con anti-windup	14
4.1.3. Control PI y overclocking	15
4.1.4. Contador y muestreo	15
4.1.5. Núcleo de SNN	15
4.2. Evaluación de ADMM en distintas plataformas	15

4.2.1.	Errores numéricos	15
4.2.2.	Caracterización tiempos de ejecución	16
5.	Conclusiones y trabajo futuro	17
5.1.	Conclusiones.....	17
5.2.	Trabajo futuro.....	17
6.	Anexos	18
6.1.	Material complementario.....	18
6.2.	Datos adicionales	18

Índice de figuras

Índice de tablas

4.1	Resumen de mediciones por caso (completar con reportes de síntesis y <i>FIL</i>). .	16
-----	--	----

1. Introducción

El trabajo presentado en esta memoria busca evaluar la utilidad de las herramientas *HDL Coder* y *HDL Verifier* de Matlab para generar y verificar código HDL a partir de descripciones de alto nivel, poniendo énfasis en la implementación en *FPGAs* y en la validación mediante *FPGA-in-the-Loop* (FIL). El foco se sitúa en documentar un flujo de trabajo reproducible que parta desde modelos en *Matlab* y *Simulink*, continúe con la generación automática de código y concluya con la verificación funcional y temporal sobre hardware objetivo. Se busca complementar la documentación oficial de Matlab con directrices prácticas y ejemplos que reflejen las restricciones de memoria, temporización y arquitectura propias de sistemas embebidos y plataformas reconfigurables.

Se espera lograr un aporte al entregar evidencia experimental que permita a investigadores e ingenieros juzgar la validez del flujo de diseño propuesto y reducir la brecha entre la descripción de alto nivel y la implementación verificable en hardware, habilitando prototipado rápido con garantías funcionales y el poder simular con sistemas continuos y mas reales. Todo el material generado (scripts, modelos, bitstreams y reportes) está organizado para que pueda ser reutilizado o extendido en futuros proyectos.

1.1. Motivación y contexto

Matlab es ampliamente utilizado en la academia y la industria para modelar, simular y analizar algoritmos. Sus entornos de programación (scripts `.m`, bloques de *Simulink*, *Stateflow*, entre otros) permiten trabajar con distintos niveles de abstracción y acceder a un vasto ecosistema de *toolboxes*. Sin embargo, la verificación funcional en un computador de escritorio no garantiza que el algoritmo pueda ejecutarse eficientemente ni de forma determinista en un sistema embebido o en una FPGA. Las abstracciones de alto nivel facilitan el desarrollo, pero ocultan detalles de recursos, latencia y temporización que resultan críticos cuando se apunta a implementaciones físicas.

Los lenguajes HDL como VHDL o Verilog, y su escritura manual exige considerar estructuras de hardware, sincronización, consumo de LUTs, registros y bloques DSP, además de la gestión de *clock domains* y reset. *HDL Coder* promete traducir modelos de alto nivel a descripciones HDL sintetizables, mientras que *HDL Verifier* habilita la validación contra el modelo original, incluyendo la co-simulación con simuladores de terceros y el uso de *FIL*. Esta última técnica permite ejecutar el algoritmo directamente en la FPGA pero manteniendo el banco de pruebas en Matlab/Simulink, reduciendo el tiempo de depuración y acercando la validación a condiciones reales; adicionalmente, abre la puerta a probar bloques HDL ya existentes (de terceros) conectándolos al entorno de referencia para contrastar funcionalidad

y temporización sin reescribir el banco de pruebas.

El desafío práctico radica en que no todo modelo de alto nivel es directamente portable a hardware: tipos de datos, dimensiones dinámicas, estructuras de control, acceso a memoria y temporización deben adecuarse a las restricciones del dispositivo. Las capas de abstracción de Matlab no relevan al diseñador de entender estos aspectos, y una configuración inadecuada puede resultar en código HDL que no sintetiza, que no cumple con las restricciones de tiempo o que altera la funcionalidad. Además, el ciclo de iteración hardware–software (generar, sintetizar, cargar, medir) es costoso si no se cuenta con una metodología clara y automatizada.

1.2. Planteamiento del problema

El objetivo central de esta memoria es diseñar, validar y documentar una metodología para llevar algoritmos descritos en Matlab/Simulink a implementaciones verificadas en FPGA mediante *HDL Coder* y *HDL Verifier*, apoyándose en *FIL* para cerrar el ciclo de verificación sobre hardware. Además, se incorpora la evaluación de bloques HDL preexistentes (desarrollados fuera de matlab) verificándolos mediante *FIL* usando el entorno de Simulink para testarlo en un entorno mas cercano a la realidad. Los objetivos específicos son:

- Definir un flujo que parta en la especificación de alto nivel y culmine en un bitstream funcional, destacando configuraciones clave de *HDL Coder* (tipos de dato fijos, gestión de recursos, *pipelining* y *resource sharing*).
- Validar la equivalencia funcional entre el modelo original y el código HDL generado mediante *HDL Verifier*, usando co-simulación y *FIL* para detectar divergencias numéricas o de temporización.
- Verificar HDL existentes en un entorno de *Simulink* que emula condiciones cercanas a un sistema real, conectando el bloque a modelos del proceso y usando *FIL*/co-simulación para confirmar su funcionalidad frente a estímulos y respuestas realistas.
- Documentar recomendaciones, limitaciones y advertencias que no aparecen de forma explícita en la documentación base, enfocadas en facilitar la extensión a nuevos algoritmos y plataformas.

1.3. Metodología general

El trabajo se estructura en cuatro etapas principales:

1. **Ajuste del modelo de alto nivel:** se preparan los algoritmos en Matlab/Simulink asegurando tipos fijos, dimensiones estáticas y estructuras de control sintetizables. Se

aplican lineamientos de *modeling for HDL* (evitar recursión, matrices dinámicas, *while* sin cotas, etc.).

2. **Generación de código HDL:** utilizando *HDL Coder*, se producen descripciones en VHDL/Verilog. Se revisan los reportes de recursos y *critical paths*, y se aplican optimizaciones (p. ej., *pipeline* y repartición de recursos) para cumplir restricciones de área y frecuencia.
3. **Verificación funcional y temporal:** con *HDL Verifier*, se realiza co-simulación y se implementa *FPGA-in-the-Loop*. Esta fase comprueba equivalencia funcional respecto al modelo de referencia y detecta desviaciones introducidas por la cuantización o por la latencia de hardware.
4. **documentación:** se miden tiempos de ejecución, utilización de recursos y throughput; se comparan con la ejecución en software y se crean guías prácticas en un repositorio GIT para su reproducción.

Cada etapa está acompañada de scripts automatizados que permiten repetir el flujo con distintas variantes de los algoritmos, registrando configuraciones de *HDL Workflow Advisor*, archivos de constraints y bancos de pruebas usados en las pruebas FIL.

1.4. Alcances y contribuciones

El trabajo se acota a la exploración de *HDL Coder* y *HDL Verifier* para la generación y verificación de código HDL orientado a FPGAs. Se tomarán funciones y modelos en Matlab/Simulink que resuelven tareas concretas (p. ej., control, procesamiento de señales, optimización) y se adaptarán al flujo para ilustrar patrones de modelamiento, problemas frecuentes y estrategias de mitigación. Además, se incorporan ejemplos que usan HDL ya existente (IP de terceros) para mostrar cómo integrarlos y validarlos con *FIL* junto a los modelos de referencia.

Las principales contribuciones son:

- Evidencia empírica sobre la capacidad de *HDL Coder* para generar código sintetizable y eficiente a partir de modelos de alto nivel, destacando sus límites prácticos.
- Un flujo reproducible de verificación con *HDL Verifier* y *FIL* que detecta divergencias funcionales y de temporización antes de comprometer un diseño a producción.
- Un repositorio con ejemplos, bitstreams, scripts de automatización y guías breves que facilitan adoptar el flujo en nuevos proyectos, reduciendo tiempos de implementación y riesgo de errores de bajo nivel.

1.5. Organización del informe

El resto del documento complementa los entregables principales y se organiza como sigue:

- El Capítulo 2 resume herramientas de generación automática de código desde alto nivel, con énfasis en *HDL Coder* y *HDL Verifier*.
- El Capítulo 3 detalla el flujo propuesto, configuraciones críticas y bancos de prueba usados en co-simulación y *FIL*, junto con las plataformas FPGA evaluadas.
- El Capítulo 4 presenta los resultados experimentales: equivalencia funcional, utilización de recursos y tiempos de ejecución comparados.
- El Capítulo 5 expone conclusiones y pautas para trabajos futuros, incluyendo oportunidades de optimización y extensiones a otros algoritmos o dispositivos.

2. Antecedentes

Este capítulo ofrece un panorama general de herramientas de generación automática de código orientadas a hardware reconfigurable. Se destacan los enfoques de *High-Level Synthesis* (HLS) para traducir descripciones de alto nivel a HDL, su presencia en la industria y sus principales aplicaciones.

2.1. Síntesis de alto nivel y herramientas de generación de código en la industria

La síntesis de alto nivel (HLS) convierte especificaciones escritas en lenguajes como C, C++, SystemC u OpenCL en descripciones RTL (VHDL o Verilog) sintetizables. Este enfoque permite a los diseñadores trabajar con abstracciones algorítmicas, delegando en la herramienta la planificación de ciclos, la asignación de recursos y la generación de pipelines o paralelismo explícito. El objetivo es reducir el tiempo de desarrollo y simplificar la exploración de arquitecturas sin escribir HDL manual, manteniendo control sobre latencia, throughput y uso de recursos [4, 7].

Para FPGAs y SoCs reconfigurables, HLS es la vía común para implementar aceleradores: Vitis HLS (AMD/Xilinx) [15], Intel HLS Compiler [10] y Catapult HLS [3] permiten describir bloques en C/C++/SystemC y obtener RTL optimizado, incluyendo pragmas para controlar *pipelining*, *unrolling* y reparto de recursos. En ecosistemas mixtos, herramientas como System Generator (integrada en Vivado) [14], LabVIEW FPGA [11] o HDL Coder [8] generan HDL a partir de diagramas de bloques y se acoplan a flujos de verificación y síntesis.

HLS facilita prototipado rápido y permite evaluar diseños bajo condiciones realistas mediante simulación, co-simulación y esquemas *hardware-in-the-loop* (HIL/FIL). Estas técnicas permiten inyectar estímulos representativos, observar respuestas con modelos físicos y detectar divergencias antes de comprometer el diseño a producción, lo que es esencial en sistemas con requisitos estrictos de temporización o seguridad.

En sectores donde la confiabilidad es crítica (automotriz, aeroespacial o industrial), los flujos HLS incorporan prácticas de trazabilidad y verificación formal para asegurar que el RTL generado respete la funcionalidad especificada. Además de las herramientas comerciales, alternativas abiertas como LegUp HLS [2] o Chisel [1] ofrecen flexibilidad para investigación y docencia, permitiendo explorar microarquitecturas, ajustar transformaciones y combinar generación automática con verificación estructurada.

Otras opciones de generación automática incluyen LabVIEW FPGA [11], que traduce diagramas VI a HDL/bitstreams con énfasis en instrumentación y adquisición de datos. La

variedad de herramientas refleja diferentes grados de integración, licenciamiento y soporte de plataformas, pero comparten el objetivo de reducir el esfuerzo de codificación RTL y acelerar el ciclo de diseño, manteniendo la capacidad de verificación temprana.

2.2. Herramientas de MATLAB para generación y verificación de código

El ecosistema de MathWorks está orientado a diseño basado en modelos y a la generación automática de código desde representaciones de alto nivel. Esta aproximación permite simular y validar algoritmos en un entorno controlado antes de traducirlos a implementaciones ejecutables en procesadores embebidos o en hardware reconfigurable. Dentro de este flujo, MATLAB/Simulink se utiliza como referencia funcional, y las herramientas de *coder* convierten esa referencia en artefactos de implementación con trazabilidad y reportes de equivalencia [12, 13, 5, 8, 9].

La ventaja principal de este enfoque es la continuidad entre modelo y ejecución: el mismo algoritmo se valida con bancos de prueba en Simulink y luego se convierte a C/C++ o HDL sin reescribirlo en un lenguaje de bajo nivel. Esto disminuye errores de traducción manual, reduce tiempos de desarrollo y permite iteraciones rápidas cuando se modifica un requisito del sistema.

2.2.1. MATLAB Coder

MATLAB Coder genera código C/C++ a partir de funciones escritas en MATLAB. Está orientado a transformar algoritmos de alto nivel en bibliotecas compilables o *MEX* para acelerar ejecuciones en PC, y también habilita despliegue en plataformas embebidas cuando se integra con toolchains específicos. En un flujo típico, el diseñador define tipos de entrada, fija tamaños de matrices y valida que el código sea compatible con generación (*codegen*). Esto obliga a declarar restricciones explícitas (dimensiones estáticas, tipos numéricos, ausencia de dinámicas no soportadas) y facilita un comportamiento determinista. MATLAB Coder se utiliza para preparar algoritmos que luego pueden migrar a Simulink o a flujos de hardware, manteniendo trazabilidad entre el modelo y el código generado [12].

2.2.2. Simulink Coder

Simulink Coder genera código C/C++ desde modelos de Simulink y está orientado a prototipado rápido, simulación acelerada y despliegue en entornos de tiempo real. El flujo permite validar el comportamiento del modelo y luego producir código estructurado que conserva la jerarquía de subsistemas y el *timing* del diagrama. En contextos de control y procesamiento digital, esto habilita transiciones directas desde el diseño de alto nivel a una implementación ejecutable, manteniendo la sincronización de muestras y facilitando pruebas *SIL/PIL* para

validar equivalencia del código generado [13].

2.2.3. Embedded Coder

Embedded Coder extiende Simulink Coder con capacidades orientadas a producción: generación de código optimizado para tiempo real, trazabilidad con requisitos, reportes de cobertura, personalización de estilos de código y compatibilidad con estándares industriales (por ejemplo, MISRA C o flujos AUTOSAR). Su rol principal es cerrar la brecha entre el prototipo funcional y el software de producción, proporcionando control sobre el uso de memoria, la estructura de datos y el *scheduler*. Esto es relevante cuando los modelos se deben ejecutar en microcontroladores o DSPs con restricciones estrictas [5].

2.2.4. HDL Coder

HDL Coder permite generar RTL (VHDL o Verilog) desde modelos de Simulink o funciones de MATLAB. El flujo se apoya en el *HDL Workflow Advisor*, que configura el tipo de objetivo, los relojes, el *oversampling* y las optimizaciones de arquitectura. Entre sus capacidades destacadas se incluyen: generación automática de *pipelines*, reparto de recursos en bucles (*resource sharing*), inserción de registros para cumplir temporización, y reportes de latencia y utilización. La herramienta exige que los modelos sean sintetizables: dimensiones estáticas, ciclos acotados y tipos de datos compatibles (preferentemente *fixed-point*) [8].

Un elemento crítico es la conversión de punto flotante a punto fijo. HDL Coder se integra con Fixed-Point Designer para proponer tamaños de palabra, evaluar error numérico y definir reglas de saturación y redondeo. Esta etapa determina el rango dinámico y la precisión, afectando directamente el uso de recursos y la equivalencia funcional. En consecuencia, la recomendación general es validar primero el modelo en punto fijo y luego generar HDL, evitando conversiones tardías que puedan introducir saturaciones no controladas [6].

Además, HDL Coder puede generar IP cores con interfaces estándar para integrarlos en flujos de diseño de FPGA. Este paso es relevante cuando el diseño se conecta a buses o a procesadores embebidos, ya que permite empaquetar el bloque como IP con interfaces configurables y documentación de puertos.

2.2.5. HDL Verifier

HDL Verifier complementa el flujo de HDL Coder con mecanismos de verificación. Permite realizar co-simulación entre Simulink y simuladores HDL de terceros, comparando la salida del modelo con la del HDL generado y reportando discrepancias. Esta etapa es especialmente útil para detectar errores de cuantización, latencias desalineadas o efectos de saturación que no aparecen en simulación puramente en software [9].

Una característica central es *FPGA-in-the-Loop* (FIL), donde el diseño HDL se sintetiza, se implementa en FPGA y se ejecuta en hardware real mientras Simulink actúa como banco de

pruebas. El modo *lockstep* sincroniza ciclos de FPGA con pasos de simulación; el modo *free-running* permite que el hardware ejecute a su reloj interno y Simulink intercambie datos bajo demanda. El ajuste del *overclocking factor* es esencial para alinear el número de ciclos internos con el muestreo de Simulink, particularmente cuando el HDL tiene latencias multiciclo.

2.2.6. Discusión

Las herramientas de *coder* conforman un flujo escalonado: MATLAB Coder y Simulink Coder se orientan a generación de C/C++ para ejecución en CPU/DSP, Embedded Coder refina ese flujo para producción embebida, y HDL Coder/HDL Verifier permiten llevar modelos al dominio RTL con verificación en hardware. En este contexto, HDL Coder y HDL Verifier son complementarios: el primero transforma el modelo a HDL y entrega reportes de arquitectura, mientras que el segundo valida equivalencia funcional mediante co-simulación y *FIL*. La calidad del resultado depende menos de la herramienta y más de cómo se modela el algoritmo (tipos de datos, límites de iteración y estructura de control), lo que confirma la importancia de aplicar buenas prácticas de *modeling for HDL* desde el inicio.

3. Metodología y desarrollo

Este capítulo describe el flujo de trabajo propuesto para usar *HDL Coder* y *HDL Verifier*, los casos de estudio seleccionados y la metodología de validación en simulación y *FIL*. El contenido complementa el material disponible en el repositorio de código, que concentra scripts, modelos y bitstreams necesarios para reproducir y extender los resultados.

3.1. Aplicaciones de ejemplo

Los ejemplos seleccionados cubren control, verificación y cómputo intensivo para validar el flujo completo sin entrar en el detalle de cada implementación. Se incluyen: producto punto (flujo base y comparación fixed vs floating), PID con anti-windup (lazo cerrado y *FIL*), controlador PI para estudiar *overclocking*, contador para caracterizar muestreo, MPC/ADMM como caso exigente en recursos, y un núcleo de SNN para evaluar procesamiento por eventos. El objetivo de esta selección es obtener conclusiones comparables sobre equivalencia funcional, latencia y uso de recursos en distintos perfiles de diseño.

3.2. Flujo de trabajo

El flujo considera: ajuste del modelo en Matlab/Simulink para tipos y estructuras sintetizables; configuración de *HDL Coder* para generación RTL; co-simulación y *FIL* con *HDL Verifier* para equivalencia funcional; y registro de configuraciones y reportes en el repositorio. Cada caso de estudio sigue la misma secuencia para facilitar la comparación entre algoritmos.

3.3. Plataformas objetivo y configuraciones

Se emplean FPGAs de distintas gamas para validar funcionalidad y latencias: placas con dispositivos Artix/Cyclone para perfiles de recursos acotados y SoC reconfigurables (por ejemplo, Zynq) para evaluar integración con procesadores embebidos. Para *FIL* se definen relojes, interfaces de E/S y restricciones mínimas necesarias para ejecutar los bancos de pruebas sin optimizaciones específicas de plataforma.

3.4. Implementaciones

Cada implementación incluye scripts de generación, archivos de proyecto y bancos de prueba. Los casos de control (PID y MPC) se verifican contra modelos de planta en *Simulink*, midiendo error de seguimiento y comportamiento bajo saturación. El núcleo SNN se valida con patrones de disparo conocidos y monitoreo de eventos para asegurar coincidencia con el modelo de referencia. Reportes de uso de recursos y frecuencia estimada se almacenan junto a los bitstreams generados.

4. Resultados experimentales

4.1. Validación funcional de códigos generados

La validación funcional se realizó comparando el comportamiento del modelo de referencia en Matlab/Simulink con el HDL generado. Se trabajó en tres niveles: simulación a nivel de modelo, co-simulación HDL y ejecución *FPGA-in-the-Loop*. En cada etapa se aplicaron estímulos representativos y pruebas de borde para verificar saturación, truncamiento y reinicio. El foco estuvo en concluir si el flujo mantiene equivalencia funcional bajo las restricciones de hardware, sin detallar la implementación de cada ejemplo.

Para evitar diferencias por cuantización, los modelos se ajustaron a *fixed-point* con escalamiento explícito. La comparación se realizó en forma *bit-accurate* cuando fue posible y, en casos con punto flotante, se emplearon tolerancias acordes al error de cuantización. En términos cualitativos, la equivalencia funcional se observó en la respuesta transitoria y estacionaria, y las discrepancias principales aparecieron en condiciones de saturación y acumuladores integrales, lo que confirma la necesidad de ajustar el rango dinámico desde etapas tempranas.

4.1.1. Producto punto (flujo base)

La generación en punto fijo mostró buena correspondencia con el modelo de referencia dentro del rango de pruebas, con errores acotados por la precisión fraccional. Activar *stream loops* reduce recursos a costa de mayor latencia, mientras que el punto flotante incrementa el uso de recursos y la latencia sin aportar beneficios claros en este ejemplo. La co-simulación y el *FIL* no mostraron errores para los datos testeados, lo que refuerza la necesidad de un banco de pruebas representativo.

4.1.2. PID con anti-windup

El flujo *FIL* replica el comportamiento del controlador en lazo cerrado con parámetros configurables y evidencia que el anti-windup reduce el sobreimpulso frente a referencias altas. La equivalencia funcional depende de mantener coherencia de saturaciones y escalamiento entre Simulink y HDL. El caso confirma que el control discreto es un candidato directo para generación automática si se cuidan los tipos de dato.

4.1.3. Control PI y overclocking

El factor de *overclocking* debe coincidir con los ciclos internos del diseño para reproducir el comportamiento del modelo base. Un factor menor degrada la respuesta; un factor muy alto introduce sobreimpulso y mayor tiempo de asentamiento. Esto confirma que el ajuste de muestreo es crítico en la validación con *FIL* cuando existe cómputo multiciclo.

4.1.4. Contador y muestreo

El ejemplo del contador permite inferir el *overclocking* necesario a partir del periodo observado en la salida. Un *overclocking* alto produce acumulación interna y valores observados mayores a los esperados, lo que muestra que el muestreo debe alinearse con la frecuencia efectiva de cada subcontador.

4.1.5. Núcleo de SNN

La SNN muestra que el flujo es viable para arquitecturas por eventos y que la equivalencia se mantiene si el *oversampling* interno se configura de acuerdo con la latencia del diseño. La comparación flotante vs fijo evidenció diferencias menores que se corrigen ajustando bits fraccionarios. Se recomienda aumentar el número de frames para elevar la confianza estadística.

4.2. Evaluación de ADMM en distintas plataformas

El caso ADMM/MPC representa el límite práctico del flujo cuando la descripción original no está orientada a sintetizabilidad. Sin optimización, el uso de recursos es excesivo y la implementación en FPGA falla por tamaño. Al reestructurar los bucles y fijar el número máximo de iteraciones, se reduce la utilización y se obtiene un HDL viable, aunque con una frecuencia interna elevada. La conclusión principal es que el algoritmo puede mantenerse funcional, pero requiere cambios explícitos en la forma de cómputo para que la herramienta explote el *resource sharing*.

4.2.1. Errores numéricos

Los errores numéricos se concentran en cuantización de coeficientes y truncamiento intermedio. En ADMM, la cuantización de matrices impacta la solución óptima y puede introducir oscilaciones leves si la escala no es adecuada. El uso de punto fijo exige validar rangos máximos para evitar saturaciones silenciosas, especialmente cuando se comparten recursos y se incrementa la latencia interna.

Para mitigar estos efectos se recomienda aplicar escalamiento por etapa, incluir márgenes de

saturación y validar con señales que cubran la dinámica completa. Con tamaños de palabra adecuados se observan diferencias acotadas y consistentes entre co-simulación y *FIL*.

4.2.2. Caracterización tiempos de ejecución

Los tiempos de ejecución se relacionan directamente con la latencia de pipeline y la frecuencia interna requerida. En ADMM, el costo está dominado por operaciones matriciales y por el número de iteraciones; al fijar el máximo de iteraciones se obtiene un tiempo determinista por muestra, con un *overclocking* alto. La implementación en FPGA entrega determinismo temporal, mientras que en software el tiempo depende del sistema operativo y de la carga.

Tabla 4.1: Resumen de mediciones por caso (completar con reportes de síntesis y *FIL*).

Caso	Frecuencia objetivo (MHz)	Latencia (ciclos)	Recursos (LUT/FF/DSP)
Producto punto	—	—	—
PID anti-windup	—	—	—
PI (overclocking)	—	—	—
Contador (overclocking)	—	—	—
SNN	—	—	—
ADMM/MPC	—	—	—

5. Conclusiones y trabajo futuro

5.1. Conclusiones

Esta memoria demuestra que *HDL Coder* y *HDL Verifier* permiten construir un flujo reproducible para llevar modelos de Matlab/Simulink a HDL sintetizable y validado en FPGA. El uso de *FIL* acorta el ciclo de verificación al mantener el banco de pruebas en Simulink y ejecutar el diseño sobre hardware real, lo que aporta evidencia práctica de equivalencia funcional y de cumplimiento temporal.

Los casos de estudio muestran que la brecha entre el modelo de alto nivel y el hardware se reduce cuando se trabaja con tipos *fixed-point*, escalamiento explícito y restricciones de diseño desde las etapas tempranas. En particular, el desempeño temporal es estable una vez llenado el pipeline, y las diferencias observadas se explican por cuantización y saturaciones mal configuradas, más que por defectos de la herramienta.

También se confirma que el diseñador conserva un rol central: la calidad del HDL generado depende de la estructura del modelo, la selección de arquitectura y la parametrización del flujo. La automatización no elimina la necesidad de comprender recursos, latencias y temporización, pero sí acelera la iteración y ofrece trazabilidad entre modelo y hardware.

5.2. Trabajo futuro

Como líneas de trabajo futuro se propone ampliar el conjunto de algoritmos evaluados y considerar dominios con exigencias de seguridad o certificación. En particular, se plantea integrar controladores con mayor dimensionalidad y sistemas de visión o comunicaciones, donde el ancho de banda y la latencia son críticos.

Se recomienda automatizar aún más el flujo mediante integración continua del repositorio, incorporando scripts que generen reportes de síntesis y tablas de resultados de forma automática. Otra línea relevante es comparar el desempeño del flujo basado en Matlab con herramientas HLS alternativas, para analizar diferencias de recursos, eficiencia y tiempos de diseño.

Finalmente, es pertinente evaluar estrategias avanzadas de cuantización y *auto-fixed-point* para minimizar errores numéricos sin sobredimensionar recursos, así como explorar plataformas heterogéneas que combinen CPU y FPGA para cargas de trabajo mixtas.

6. Anexos

6.1. Material complementario

El repositorio asociado a esta memoria contiene los modelos, scripts y reportes necesarios para reproducir los resultados. En particular, se incluyen:

- Modelos de Simulink de cada caso de estudio, con variantes en *fixed-point* y configuraciones de *HDL Coder*.
- Scripts de Matlab para generación automática de HDL, ejecución de co-simulación y configuración de *FIL*.
- Archivos de proyecto y restricciones para las plataformas objetivo.
- Reportes de síntesis y utilización de recursos, junto con bitstreams generados.

El objetivo del material complementario es permitir que el lector replique el flujo completo y evalúe variaciones del diseño sin reestructurar los modelos base.

6.2. Datos adicionales

Para facilitar la comparación entre versiones, se mantuvieron registros de configuración del *HDL Workflow Advisor*, versiones de Matlab/Simulink y parámetros de hardware. Adicionalmente, se documentan las señales de prueba utilizadas en simulación y los criterios de aceptación para equivalencia funcional.

Se recomienda mantener la trazabilidad de cambios en el repositorio y actualizar las tablas de resultados cuando se incorporen nuevos algoritmos o plataformas. Esto permite construir una base de evidencia acumulativa y comparar de forma consistente el impacto de nuevas configuraciones o mejoras en los modelos.

Bibliografía

- [1] Jonathan Bachrach et al. «Chisel: Constructing hardware in a Scala embedded language». En: *Design Automation Conference (DAC)*. 2012. URL: <https://doi.org/10.1145/2228360.2228584>.
- [2] Andrew Canis et al. «LegUp: High-level synthesis for FPGA-based processor/accelerator systems». En: *International Symposium on Field-Programmable Gate Arrays (FPGA)* (2011). URL: <https://doi.org/10.1145/1950413.1950430>.
- [3] *Catapult High-Level Synthesis*. Siemens EDA. 2024. URL: <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>.
- [4] Philippe Coussy y Adam Morawiec. *High-Level Synthesis: From Algorithm to Digital Circuit*. Springer, 2009. URL: <https://link.springer.com/book/10.1007/978-1-4020-8588-8>.
- [5] *Embedded Coder*. MathWorks. 2024. URL: <https://www.mathworks.com/products/embedded-coder.html>.
- [6] *Fixed-Point Designer*. MathWorks. 2024. URL: <https://www.mathworks.com/products/fixed-point-designer.html>.
- [7] Olivier Gaide y Philippe Coussy. «High-level synthesis: A decade of progress». En: *Design Automation Conference (DAC)*. 2011. URL: <https://doi.org/10.1145/2024724.2024734>.
- [8] *HDL Coder*. MathWorks. 2024. URL: <https://www.mathworks.com/products/hdl-coder.html>.
- [9] *HDL Verifier*. MathWorks. 2024. URL: <https://www.mathworks.com/products/hdl-verifier.html>.
- [10] *Intel High Level Synthesis Compiler*. Intel. 2024. URL: <https://www.intel.com/content/www/us/en/docs/programmable/683090>.
- [11] *LabVIEW FPGA Module*. NI. 2024. URL: <https://www.ni.com/en/shop/software/products/labview-fpga-module.html>.
- [12] *MATLAB Coder*. MathWorks. 2024. URL: <https://www.mathworks.com/products/matlab-coder.html>.
- [13] *Simulink Coder*. MathWorks. 2024. URL: <https://www.mathworks.com/products/simulink-coder.html>.
- [14] *System Generator for DSP*. AMD Xilinx. 2024. URL: <https://www.xilinx.com/products/design-tools/vivado/integration/esl-design.html>.
- [15] *Vitis High-Level Synthesis*. AMD Xilinx. 2024. URL: <https://docs.xilinx.com/r/en-US/ug1399-vitis-hls>.